

PRUEBA TÉCNICA

INGENIERO CLOUD SENIOR

PLATAFORMA KUBERNETES CLOUD-NATIVE · GATEWAY API · GITOPS · OBSERVABILIDAD

Kubernetes · kubeadm / k3s · Gateway API v1.4 · Istio · ArgoCD · CloudNativePG · Longhorn · cert-manager
· Ngrok · Prometheus · Grafana · Jaeger · Kiali · Docker Registry

Esta prueba evalúa la capacidad del candidato para diseñar, desplegar y operar una plataforma Kubernetes cloud-native de nivel Senior. Se espera que el candidato resuelva cada bloque de forma autónoma consultando la documentación oficial de cada herramienta.

El foco de la evaluación no es la memorización de comandos, sino la comprensión de la arquitectura y la capacidad de tomar decisiones justificadas.

El tiempo de entrega es de 5 días hábiles. El candidato debe entregar un repositorio Git privado con todos los manifiestos y una guía de instalación reproducible, y estar disponible para una sesión de review en vivo de 45–60 minutos.

Versión 2.0 · Mayo 2026 · Uso Interno · Área de Ingeniería de Plataforma

RESUMEN DE BLOQUES

La siguiente tabla resume los bloques de trabajo, herramientas principales y peso de evaluación. Los bloques marcados como Senior son los de mayor exigencia y deben completarse sin omisiones.

#	BLOQUE	HERRAMIENTA PRINCIPAL	PESO	NIVEL
1	Bootstrap del cluster	kubeadm o k3s · CNI · MetalLB	12 %	Senior
2	TLS local	cert-manager · CA self-signed	8 %	Base
3	Gateway API (reemplaza Ingress)	Gateway API v1.4 · GatewayClass Istio	12 %	Senior
4	GitOps con ArgoCD	ArgoCD · Helm · App of Apps	13 %	Senior
5	Registry local (ECR simulado)	Docker Registry v2 / Zot	7 %	Base
6	Aplicación ToDo – 2 microservicios	Go o Node.js · REST API	13 %	Senior
7	Base de datos PostgreSQL	CloudNativePG	10 %	Senior
8	Almacenamiento persistente	Longhorn / OpenEBS	8 %	Base+
9	Observabilidad con Istio	Istio · Jaeger · Kiali · Prometheus · Grafana	12 %	Senior
10	Exposición pública	Ngrok Operator / cloudflared	5 %	Base

Los bloques "Base" deben implementarse siguiendo la documentación oficial paso a paso. Los bloques "Senior" requieren que el candidato entienda el propósito de cada recurso y sea capaz de explicarlo y depurarlo durante el review.

BOOTSTRAP DEL CLUSTER KUBERNETES

► ELECCIÓN DE LA HERRAMIENTA DE BOOTSTRAP

El cluster debe levantarse sobre máquinas virtuales o bare-metal utilizando una de las dos opciones indicadas a continuación. No se acepta kind ni ninguna solución basada en contenedores Docker como nodos del cluster (minikube, k3d, etc.). La elección debe justificarse en el README con criterios técnicos claros.

OPCIÓN	DISTRIBUCIÓN	PERFIL DE USO	CONSIDERACIONES CLAVE
A	kubeadm (upstream)	Cluster vanilla, máximo control, mayor fidelidad a producción enterprise	Requiere configurar CNI, MetalLB y StorageClass manualmente. Exige al menos 2 VMs (1 control-plane + 1 worker). Proceso más largo pero más educativo.
B	k3s (Rancher / CNCF)	Distribución ligera certificada, recomendada para entornos con recursos limitados	Incluye containerd y un load balancer básico. Instala más rápido. Usar etcd como backend de datos (no SQLite) para simular prácticas de producción.

► REQUISITOS DEL CLUSTER (INDEPENDIENTE DE LA OPCIÓN ELEGIDA)

- Mínimo 1 nodo control-plane + 1 nodo worker. Se recomiendan 2 workers para que Longhorn pueda replicar volúmenes entre nodos.
- El CNI instalado debe ser compatible con Istio. Se recomienda Calico o Cilium; Flannel es compatible pero con funcionalidades de NetworkPolicy limitadas.
- Instalar MetalLB en modo Layer 2 para proveer servicios de tipo LoadBalancer en bare-metal. Esto es necesario para que el Gateway de Istio obtenga una IP externa. En k3s, el load balancer integrado puede usarse como alternativa justificada.
- Verificar que todos los nodos están en estado Ready y el CNI operativo antes de continuar con los siguientes bloques.

GATEWAY API V1.4 — REEMPLAZA INGRESS

▸ CONTEXTO Y MOTIVACIÓN

La Kubernetes Gateway API v1.4 (GA, octubre 2025) es el sucesor oficial del recurso Ingress y la forma recomendada para gestionar el tráfico north-south en clusters modernos. A diferencia de Ingress, que era un recurso monolítico controlado por anotaciones específicas de cada controlador, Gateway API introduce una separación de roles clara:

RECURSO	RESPONSABLE	FUNCIÓN
GatewayClass	Admin de infraestructura	Define el tipo de implementación del gateway (p.ej. Istio como controlador).
Gateway	Equipo de plataforma	Declara los listeners (puertos, protocolos, TLS) y políticas de red del punto de entrada.
HTTPRoute	Equipo de aplicación	Define reglas de enrutamiento: hosts, paths, pesos, headers, backends.
ReferenceGrant	Equipo de seguridad	Autoriza referencias cross-namespace (p.ej. Gateway accede a Secret en otro namespace).

▸ IMPLEMENTACIÓN CON GATEWAYCLASS DE ISTIO

La implementación del Gateway API se realizará usando Istio como controlador. Cuando Istio está instalado, registra automáticamente una GatewayClass llamada istio. Al crear un recurso Gateway con gatewayClassName: istio, Istio provisiona un Deployment y un Service de tipo LoadBalancer para manejar el tráfico de entrada, unificando el plano de control del service mesh con el gateway de entrada.

El candidato debe instalar primero los CRDs del Gateway API Standard Channel (v1.4.x) antes de instalar Istio. El canal Standard incluye los recursos estables: GatewayClass, Gateway, HTTPRoute, GRPCRoute y ReferenceGrant en versión v1. No usar el canal Experimental salvo que sea estrictamente necesario.

▸ DISEÑO DE GATEWAYS REQUERIDOS

- Gateway principal para la aplicación ToDo: listener HTTP en puerto 80 con redirect automático a HTTPS, y listener HTTPS en puerto 443 con TLS terminado usando el certificado emitido por la CA local de cert-manager.

- Gateway de plataforma (puede compartir el mismo Gateway con listeners adicionales) para exponer ArgoCD, Grafana, Kiali, Jaeger y Longhorn UI en sus respectivos hostnames locales con TLS.
- Las HTTPRoutes de cada aplicación deben vivir en el namespace de la aplicación y referenciar el Gateway mediante parentRefs. Usar ReferenceGrant donde sea necesario para autorizar referencias cross-namespace.
- El candidato debe explicar durante el review la diferencia entre terminación TLS en el Gateway versus en la HTTPRoute, y por qué se optó por un enfoque u otro.
- Todos los servicios expuestos (todo-frontend, todo-api, ArgoCD, Grafana, Kiali, Jaeger, Longhorn) deben tener certificados HTTPS válidos firmados por la CA local.
- El candidato debe documentar cómo importar la CA raíz en el sistema operativo o navegador para acceder sin advertencias TLS.

▸ INTEGRACIÓN CON CERT-MANAGER

cert-manager gestiona los certificados TLS del cluster. Debe instalarse con una CA raíz local auto-firmada usando dos ClusterIssuers encadenados: uno de tipo selfsigned para arrancar, y otro de tipo CA para emitir los certificados finales. Los Gateways referencian los Secrets de certificados generados por cert-manager en sus listeners HTTPS.

GITOPS CON ARGOCD

▸ INSTALACIÓN Y ACCESO

Instalar ArgoCD en el namespace argocd usando el manifiesto oficial o el chart de Helm. La UI debe ser accesible mediante una HTTPRoute del Gateway API apuntando al hostname argocd.local con TLS de la CA local. No se acepta acceso exclusivo por port-forward durante el review.

▸ ESTRUCTURA DEL REPOSITORIO GITOPS

El repositorio Git actuará como única fuente de verdad siguiendo el patrón App of Apps: una Application raíz gestiona el resto de Applications como recursos declarativos. Estructura mínima esperada:

- Directorio apps/ con una Application por microservicio (todo-api y todo-frontend), apuntando a su carpeta de manifiestos.
- Directorio infra/ con Applications para: cert-manager, Gateway API CRDs, Longhorn, CloudNativePG y MetalLB.
- Directorio observability/ con Applications para: Istio, kube-prometheus-stack, Loki y la configuración de Gateways de la plataforma.

▸ REQUISITOS DE COMPORTAMIENTO

- El README debe documentar cómo registrar el repositorio en ArgoCD y cómo aplicar la Application raíz para disparar el sync inicial.
- Todas las Applications deben mostrar estado Synced + Healthy al momento del review.
- Demostrar un ciclo GitOps completo: realizar un cambio en el repositorio (p.ej. modificar el número de réplicas) y verificar que ArgoCD detecta la diferencia y aplica el cambio automáticamente.
- Usar Sync Waves para garantizar el orden correcto: CRDs del Gateway API y cert-manager primero, luego Istio, luego base de datos, finalmente la aplicación.
- Los manifiestos de HTTPRoutes y Gateways deben estar versionados en el repositorio y gestionados por ArgoCD, no creados manualmente.

REGISTRY LOCAL — ECR SIMULADO

▸ IMPLEMENTACIÓN DEL REGISTRY

El candidato debe desplegar un registry de contenedores dentro del cluster que simule el comportamiento de un ECR privado. La elección debe justificarse en el README:

REGISTRY	TIPO	OCI	COMPLEJIDAD
Docker Registry v2	Deployment K8s	Parcial	Baja — recomendada para este nivel
Zot Registry	Helm chart	Completo (OCI Distribution Spec)	Media — valorado positivamente
Harbor	Helm (complejo)	Completo	Alta — opcional / bonus

El registry debe ser accesible desde los nodos del cluster para que Kubernetes pueda hacer pull de las imágenes. El flujo esperado es: construir la imagen localmente, hacer push al registry del cluster, y referenciar esa imagen en los Deployments.

▸ INTEGRACIÓN CON ECR REAL DE AWS (OPCIONAL)

Como alternativa al registry local, se puede usar ECR real en AWS. En ese caso debe resolverse el problema de la expiración del token de autenticación (12 horas), ya sea con un CronJob de refresco o con un operador dedicado. Esta opción suma puntos adicionales.

APLICACIÓN TODO – DOS MICROSERVICIOS

▸ DESCRIPCIÓN

La aplicación constará de dos microservicios en el namespace todo, ambos con sidecar de Istio inyectado. La comunicación del frontend hacia la API debe fluir a través del Gateway de Istio (HTTPRoute), de modo que cada request sea capturado como traza en Jaeger.

MICROSERVICIO	TECNOLOGÍA	PUERTO	RESPONSABILIDAD
todo-api	Go 1.22 o Node.js 20	8080	API REST CRUD de tareas. Conecta a PostgreSQL. Expone /healthz, /readyz y /metrics.
todo-frontend	React 18 + Nginx	3000	SPA que consume todo-api. Permite crear, listar, editar y eliminar tareas.

▸ ENDPOINTS REQUERIDOS EN TODO-API

- GET /healthz — Liveness probe. Responde 200 OK siempre que el proceso esté vivo.
- GET /readyz — Readiness probe. Responde 200 OK solo cuando la conexión a la base de datos está establecida.
- GET /metrics — Métricas en formato Prometheus (librería cliente del lenguaje elegido).
- GET /tasks — Listar todas las tareas almacenadas en PostgreSQL.
- POST /tasks — Crear una tarea. Body JSON: { title, done }.
- GET /tasks/{id} — Obtener una tarea por ID.
- PUT /tasks/{id} — Actualizar título o estado de una tarea.
- DELETE /tasks/{id} — Eliminar una tarea.

▸ RECURSOS KUBERNETES REQUERIDOS

RECURSO	REQUISITO MÍNIMO	PROPÓSITO A EVALUAR
Deployment	2 réplicas, RollingUpdate	Disponibilidad durante actualizaciones sin downtime
HTTPRoute		

RECURSO	REQUISITO MÍNIMO	PROPÓSITO A EVALUAR
	Una HTTPRoute por microservicio apuntando al Gateway principal	Enrutamiento declarativo con Gateway API, sin Ingress clásico
HorizontalPodAutoscaler	CPU target 60 %, min 2, max 5	Escalado automático ante carga
Liveness y Readiness Probes	HTTP hacia /healthz y /readyz respectivamente	Kubernetes gestiona el ciclo de vida del pod correctamente
Resources requests/limits	CPU y Memoria definidos explícitamente	Evitar consumo ilimitado de recursos del nodo
NetworkPolicy	Solo frontend → API; solo API → PostgreSQL	Segmentación de red dentro del namespace
ConfigMap + Secret	Variables de entorno externalizadas (host DB, credenciales)	Configuración separada del código, secretos protegidos
ServiceAccount	Uno por microservicio	Principio de mínimo privilegio en RBAC

BASE DE DATOS — CLOUDNATIVEPG

▸ INSTALACIÓN DEL OPERADOR

CloudNativePG (cloudnative-pg.io) es el operador PostgreSQL recomendado para Kubernetes. Gestiona el ciclo de vida completo: creación de instancias, alta disponibilidad, failover automático, backups y monitoreo nativo con Prometheus. Debe instalarse en el namespace `cnpg-system` antes de definir el recurso Cluster.

▸ DEFINICIÓN DEL CLÚSTER POSTGRESQL

- 3 instancias: 1 primaria y 2 réplicas de lectura.
- La base de datos inicial debe llamarse `tododb` con un usuario propietario cuyas credenciales se almacenen en un Secret de Kubernetes.
- El almacenamiento de cada instancia debe usar la StorageClass de Longhorn, con un tamaño mínimo de 5Gi por instancia.
- Habilitar el PodMonitor integrado de CloudNativePG para que Prometheus recoja métricas de PostgreSQL automáticamente.

▸ CONEXIÓN DESDE LA APLICACIÓN

CloudNativePG genera automáticamente Services de Kubernetes para los distintos roles del clúster (lectura/escritura y solo lectura). La aplicación `todo-api` debe conectarse al Service RW generado por el operador. Las credenciales deben provenir del Secret creado por CloudNativePG, inyectado como variables de entorno en el Deployment.

▸ VALIDACIONES ESPERADAS DURANTE EL REVIEW

- El clúster debe reportar estado `Healthy` con las 3 instancias activas.
- Demostrar failover automático: eliminar el pod de la instancia primaria y verificar que una réplica asuma el rol en menos de 30 segundos, sin intervención manual.
- Las tareas creadas desde el frontend deben persistir al reiniciar los pods de `todo-api`.
- Las métricas de PostgreSQL deben ser visibles en el dashboard de Grafana.

ALMACENAMIENTO PERSISTENTE — LONGHORN

▸ JUSTIFICACIÓN Y REQUISITOS

Longhorn (CNCF) es el sistema de almacenamiento distribuido recomendado para este despliegue. Provee volúmenes block replicados dentro del cluster, snapshots, backups y una UI de gestión web. Su función es garantizar que los datos de PostgreSQL sobrevivan al reinicio de pods, nodos y del propio cluster. Es la mejor práctica para clusters on-prem o bare-metal sin almacenamiento cloud administrado.

▸ CONSIDERACIONES DE INSTALACIÓN

Longhorn requiere open-iscsi instalado en los nodos del cluster. En kubeadm o k3s sobre VMs estándar (Ubuntu 24.04 LTS), esto se instala a nivel de sistema operativo en cada nodo antes de desplegar Longhorn.

▸ VALIDACIONES ESPERADAS

- Longhorn debe instalarse antes de desplegar CloudNativePG.
- Configurar un mínimo de 2 réplicas por volumen para garantizar disponibilidad ante la pérdida de un nodo.
- La UI de Longhorn debe ser accesible mediante una HTTPRoute del Gateway API (longhorn.local) con TLS.
- Si la instalación de open-iscsi presenta dificultades, se acepta OpenEBS con Jiva o LocalPV como alternativa. La elección debe justificarse en el README.
- Los PVC del clúster CloudNativePG deben estar en estado Bound.
- Los volúmenes deben aparecer como replicados y en estado Healthy en la UI de Longhorn.
- Demostrar que los datos persisten tras eliminar y recrear pods de PostgreSQL.

OBSERVABILIDAD – ISTIO Y STACK DE MONITOREO

▸ VISIÓN GENERAL DEL STACK

Istio es el componente central: inyecta un proxy Envoy sidecar en cada pod del namespace todo, capturando métricas L7, trazas distribuidas y aplicando políticas de tráfico sin modificar el código de la aplicación. Al usar Gateway API con GatewayClass istio, el service mesh ya está instalado como prerequisite del bloque 3, por lo que la observabilidad es una extensión natural del mismo plano de control.

CAPA	HERRAMIENTA	FUNCIÓN	INSTALACIÓN
Service mesh + Gateway	Istio 1.21+	mTLS, trazas, métricas L7, control de tráfico, implementa GatewayClass istio	istioctl (perfil demo)
Trazas distribuidas	Jaeger	Visualizar spans de cada request HTTP correlacionados entre servicios	Incluido en perfil demo de Istio
Visualización del mesh	Kiali	Grafo de dependencias y métricas de tráfico en tiempo real	Incluido en perfil demo de Istio
Métricas de infraestructura	Prometheus	Scraping de métricas de cluster, pods, app y CNPG	kube-prometheus-stack (Helm)
Dashboards	Grafana	Visualización con dashboards Istio, Kubernetes, CNPG y Longhorn	kube-prometheus-stack (Helm)
Logs centralizados	Loki + Promtail	Agregación de logs de todos los pods accesible desde Grafana	Helm chart loki-stack

▸ REQUISITOS DE INSTALACIÓN DE ISTIO

- Instalar usando el perfil demo, que incluye Jaeger, Kiali, Prometheus y Grafana preconfigurados.
- Habilitar la inyección automática de sidecar en el namespace todo. Los pods existentes deben reiniciarse para recibir el proxy Envoy.

- Verificar que todos los pods del namespace todo muestran 2/2 contenedores Ready (aplicación + sidecar Envoy).
- Todos los servicios de plataforma (Grafana, Kiali, Jaeger, ArgoCD, Longhorn) deben ser accesibles mediante HTTPRoutes del Gateway de Istio, no mediante port-forwards durante el review.

► **TRAZAS DISTRIBUIDAS CON JAEGER**

Para que las trazas muestren el recorrido completo de un request, la aplicación todo-api debe propagar las cabeceras de tracing de Istio (x-request-id, x-b3-traceid, x-b3-spanid, x-b3-sampled) en todas sus llamadas HTTP salientes. Sin esta propagación, Jaeger mostrará trazas fragmentadas sin correlación entre servicios.

- Cada endpoint (GET /tasks, POST /tasks, etc.) debe aparecer como operación diferenciada en Jaeger con al menos 2 spans por request.
- Las trazas deben mostrar correlación completa: Gateway → todo-api → PostgreSQL.
- Jaeger UI debe ser accesible mediante HTTPRoute del Gateway con TLS.

► **KIALI — VISUALIZACIÓN DEL MESH**

- Grafo de dependencias con animación de tráfico en tiempo real para el namespace todo.
- Métricas de latencia p50 y p95 visibles por servicio.
- Al menos un VirtualService configurado en el namespace todo (p.ej. timeout o retry policy en todo-api), con explicación de su propósito.

► **PROMETHEUS Y GRAFANA**

Instalar mediante el chart kube-prometheus-stack. Grafana debe ser accesible mediante HTTPRoute con TLS (grafana.local). Los siguientes dashboards deben estar importados con datos reales durante el review:

- Istio Mesh Dashboard (ID oficial 7639): RPS, error rate y latencia p95.
- Kubernetes Cluster Dashboard (ID 315): uso de CPU y Memoria por nodo y pod.
- CloudNativePG Dashboard (repositorio oficial CNPG): queries/seg, conexiones activas.

EXPOSICIÓN PÚBLICA — NGROK

▸ OBJETIVO

El candidato debe exponer la aplicación ToDo (frontend) de forma pública y accesible desde internet sin necesidad de cambios en la infraestructura de red local, para permitir una demostración remota compartiendo una URL temporal.

▸ OPCIONES DE IMPLEMENTACIÓN

- Ngrok Kubernetes Operator (recomendado): gestiona tunnels como recursos nativos de Kubernetes. Se integra con el flujo GitOps y no requiere ejecutar procesos en el host.
- Ngrok binario con port-forward: opción más sencilla, válida para la demo aunque no integrada en el cluster.
- Cloudflare Tunnel (cloudflared): alternativa con TLS real de Cloudflare y soporte para múltiples dominios simultáneos de forma gratuita.

▸ REQUISITOS DURANTE EL REVIEW

- La aplicación ToDo debe ser accesible desde una URL pública compartida por el candidato al inicio de la sesión.
- Demostrar la creación de una tarea desde el navegador usando la URL pública y verificar que persiste en PostgreSQL.

La cuenta gratuita de Ngrok permite 1 tunnel simultáneo y la URL cambia en cada sesión. Esto es suficiente para la demo. Documentar el comportamiento en el README.

ENTREGABLES Y EVALUACIÓN

► **REPOSITORIO GIT**

- Repositorio privado en GitHub o GitLab con acceso concedido al evaluador antes de la sesión.
- README.md con: prerequisites del sistema, pasos para levantar el entorno desde cero, decisiones de diseño en cada bloque, listado de hostnames a añadir en /etc/hosts, y justificación de la elección entre kubeadm y k3s.
- Todos los manifiestos versionados en el repositorio. No se aceptan recursos imperativos sin su equivalente declarativo.
- Script bootstrap.sh o Makefile con targets por bloque: cluster, infra, gateway, argocd, observability, expose.
- Historial de commits organizado y descriptivo.

► **CHECKLIST DE REVIEW EN VIVO (45–60 MIN)**

#	DEMOSTRACIÓN	EVIDENCIA ESPERADA	PESO
1	Cluster operativo	Todos los nodos Ready. CNI y MetalLB funcionales.	7 %
2	Gateway API funcional	GatewayClass istio con Accepted=True. Gateways en estado Programmed=True.	8 %
3	ArgoCD accesible via HTTPRoute + HTTPS	Todas las Applications Synced + Healthy.	8 %
4	Ciclo GitOps completo	Cambio en Git → sync automático → cambio visible en cluster.	7 %
5	Crear tarea desde el frontend	Tarea creada visible en UI y persistida en PostgreSQL.	10 %
6	Failover automático de CNPG	Eliminar pod primario → nueva primaria activa en menos de 30 s.	12 %
7			13 %

#	DEMOSTRACIÓN	EVIDENCIA ESPERADA	PESO
	Trazas en Jaeger via HTTPRoute	Span completo visible para GET /tasks y POST /tasks.	
8	Topología en Kiali via HTTPRoute	Grafo animado con métricas de latencia del namespace todo.	10 %
9	Dashboards en Grafana via HTTPRoute	Istio Mesh + CNPG dashboards con datos reales.	10 %
10	Volúmenes en Longhorn via HTTPRoute	PVCs Bound y volúmenes replicados en UI Longhorn.	8 %
11	URL pública via Ngrok	App accesible desde internet. Tarea creada desde URL pública.	7 %

Durante el review el candidato debe poder explicar el propósito de cada componente instalado, describir qué ocurriría si se eliminara, y razonar sobre las decisiones de diseño tomadas, incluyendo la elección entre kubeadm y k3s y el uso de Gateway API sobre Ingress clásico. La consulta de documentación durante la sesión es completamente válida y esperada.

PUNTOS BONUS

Los siguientes ítems son opcionales y suman puntuación adicional. Se recomienda abordar al menos uno si el tiempo lo permite.

ÍTEM	DESCRIPCIÓN	PTS
mTLS estricto con Istio	Aplicar PeerAuthentication STRICT en el namespace todo para forzar cifrado mTLS en toda la comunicación interna gestionada por Istio.	+5
Backup de PostgreSQL a MinIO	Configurar backup automático de CloudNativePG hacia un bucket MinIO desplegado en el cluster (compatible con S3). Demostrar restauración funcional.	+5
Prueba de carga con HPA	Generar carga sobre todo-api usando k6 o hey y demostrar que el HPA escala los pods automáticamente ante la demanda.	+5
GRPCRoute en Gateway API	Demostrar enrutamiento gRPC nativo usando GRPCRoute del canal Standard de Gateway API v1.4 entre al menos dos servicios internos.	+5
Helm chart propio	Empaquetar todo-api como un Helm chart con values.yaml parametrizable (imagen, réplicas, recursos, variables de entorno).	+3
Logs con Loki	Mostrar queries de logs de todo-api y todo-frontend desde Grafana con LogQL básico.	+3
Políticas con Kyverno	Instalar Kyverno y definir al menos 2 políticas: prohibir imágenes con tag :latest y requerir que todos los pods tengan limits definidos.	+3

ÍTEM	DESCRIPCIÓN	PTS
ECR con refresco automático	CronJob que renueva el ImagePullSecret de ECR antes de que expire el token de autenticación (12 h).	+3

RECURSOS DE REFERENCIA

La consulta de documentación durante la prueba y el review es válida y esperada para un perfil Senior.

HERRAMIENTA / SPEC	DOCUMENTACIÓN OFICIAL
Kubernetes Gateway API v1.4	gateway-api.sigs.k8s.io
Gateway API – Implementaciones	gateway-api.sigs.k8s.io/implementations
Istio – Gateway API	istio.io · Gateway API
Istio – Propagación de cabeceras tracing	istio.io · Distributed Tracing
kubeadm – Crear cluster	kubernetes.io · kubeadm
k3s – Instalación	docs.k3s.io
MetalLB	metallb.universe.tf
Calico CNI	docs.tigera.io
CloudNativePG	cloudnative-pg.io/docs
Longhorn	longhorn.io/docs
cert-manager	cert-manager.io/docs
ArgoCD	argo-cd.readthedocs.io
Kiali	kiali.io/docs
Jaeger	jaegertracing.io/docs
kube-prometheus-stack	github.com/prometheus-community/helm-charts
Ngrok Kubernetes Operator	ngrok.com/docs · k8s
OpenEBS (alternativa Longhorn)	openebs.io/docs